
AliveCorKit Android Documentation

REVISION HISTORY	4
OVERVIEW	6
AliveCorKitLite	6
AliveCorKit	6
Localization	6
CONFIGURATION AND USAGE	7
Import AAR files	7
Add Dependencies	8
For AliveCorKit	9
For AliveCorKitLite	10
Configure ProGuard	11
Initialize the SDK Modules	11
Feature Access and Expiration	14
Starting from AliveCorKit SDK 1.6.2 API:	14
Launch the Default Recording Flow	14
AliveCorKitLite	15
Create a Custom Recording Flow with Premade UI Components	16
Starting RecordEkgFragment:	17
Starting the ResultEkgFragment:	17
Using EcgResultView	18
Display the Trace for a Recorded ECG	18
Generate PDF Reports for Recorded ECGs	20
Use the Encryption Password Prompt	20
class AliveCorPdfHelper	22
Properties / Methods	22
Description	22
Analyze Data Recorded from Other Sources	23
Login and Manage ECG Data with AliveCorKit	23
Login/Logout with a Kardia Pro Account	24
Interact with the Logged-In Team Member Profile	25
Validate Patient IDs for a Team	26
Store and Upload ECG Recordings	26
Load ECG Records from Storage	27
Save a New ECG Recording	27

Upload ECG Recordings to AliveCor Servers	28
API REFERENCE	29
AliveCorKitLite Classes	29
com.alivecor.api	29
class AliveCorKitLite	29
Public Methods	30
Classes Shared Between AliveCorKit and AliveCorKitLite	32
com.alivecor.api	32
enum AliveCorDevice	33
Values	33
class EkgAnalyzer	33
Methods	33
or	34
class EkgAnalyzer.Result	34
Properties:	34
class AliveCorEcg	34
Properties	34
class RecordingDeviceInfo	35
Properties	36
class RecordingConfiguration	36
Properties	37
class EcgEvaluation	38
Properties	38
enum AliveCorDetermination	39
enum LeadConfiguration	40
Values	40
com.alivecor.view	40
class EcgResultView	40
Public Methods	40
class RhythmStripView	40
Public Methods	40
com.alivecor.ecg.record	41
class RecordEkgFragment	41
Public Methods	41
interface RecordEkgListener	43
Methods	43
enum RecordingError	44
Methods	44
Values	45
class ResultEkgFragment	46

Methods	46
interface ResultEkgListener	46
Methods	47
enum ResultEkgError	47
Values	47
AliveCorKit Classes	47
com.alivecor.api	47
class AliveCorKit	47
Methods	47
class AliveCorKitUtil	50
Constants	50
Methods	51
com.alivecor.pro	51
enum AliveCorServer	51
Values	51
class EcgWebEntiity	52
Properties	52
class EcgWebEntity.AlgorithmDetail	53
Properties	53
class KardiaEcgEntity	53
Properties	53
class KardiaProEcgBackend	55
Methods	55
class KardiaProEcgRepository	56
Methods	56
class TeamMember	57
Properties	57
class TeamMember.Profile	57
class TeamMember.Team	57
Class TeamMember.Team.ScreeningConfig	57
Properties	57

REVISION HISTORY

Version	Date	Details
	1/22/2020	Documentation for v0.1 of SDKs
	5/21/2020	Update Documentation for Kit v0.5
	5/29/2020	Add Documentation for EcgResultView, Guides for Custom Recording Fragments/Views
	6/12/2020	Add Docs for App Name, new Properties on EcgResultView, Asynchronous/Encrypted PDF creation, duration property for AliveCorEcg, Cleanup, Revise docs for EcgEvaluation
1.0.0	October 19, 2020	SDK provisioning & reporting, upload events. Add Expiration logic and server choice for AliveCorKitLite
1.0.1	February 25, 2021	Fix translation for aVR and repeated leads on standard PDF filter
1.0.2	March 23, 2021	Update UniversalMonitor 1.2.7, Kardia Ai Version 1.2.4.1
1.0.3	October 28, 2021	Fix a bug with device enum, AliveCorKit.Initialize async conditions. Add ECG Frame callback in RecordEkgListener
1.0.4	March 24, 2022	Update project dependency, add secured preferences
1.0.5	March 25, 2022	Add two new url overrides to RecordingHelpUrlsRecordEkgListener Add BPM callback in the custom flow
1.0.6	May 20, 2022	Add force "Auto invert" ECG option in PDF generator
1.0.7	June 29, 2022	Update Public SDK to Android 12, API 32 support. Make help urls configurable in the default recording flow Fix PDF file size.
1.0.8	July 19, 2022	Improve BluetoothOffErrorFragment to Enable BT request. Fix crashes on the SDK launch
1.0.9	September 23, 2022	Update KardiaAI 1.2.4.2 Android 30 support Update build config and scripts
1.1.0	October 24, 2022	Fix Korean PDFs size, truncation issue. Update get SDK version method to static.

1.2.x	May-October, 2022	Add KardiaMobile Card integration and Kardia AI V2 support. Add Android 13 support. Bug fixes.
1.3.0	November 7, 2022	Special EU build based on the 1.1.0 with bug fixes.
1.3.1	February 6, 2023	Merged 1.2.x features into Public SDK Updated build scripts to support internal and external products. Bug fixes.
1.4.0	March 2, 2023	Version bump to major SDK version 1.4.0
1.5.0	Jul 11, 2023	Location permission requests optimized for Android 12 and newer PDF reports updated to the Android native API, bug fixes. Add BT Pairing canceled callback to the RecordEkgListener. Add EXTRA_ENABLE_LEADS_BUTTONS for Recording Fragment. Add AliveCorKitLite.showAboutFragment(this); Bug fixes.
1.5.1	Nov 29, 2023	Few Chinese translation updated, BT permission removed for the KardiaMobile device on Samsung Galaxy A50.
1.5.1.1	Oct 15 2024	Hot fix SDK init and crash on the LENOVO K10, Android 12 devices
1.5.5	Jul 2 2024	BugFix to Init SDK with multiple threads
1.6.0	May 3 2024	Add support for KardiaRx in EU, Add 1.3.1 EU changes (MDD SDK), Update translations
1.6.2	Oct 1 2024	Multi language support for recording and PDF reports Add Welsh language Update Kardia AI 2.0.8 Update to Android 35 Update sample app for language selection Bug fixes
1.6.3	Oct 28 2024	Minor bug fixes.
1.6.4	Jan 15 2025	Update KAI 2.0.9 and Universal Monitor 1.3.4 KM Recording screen update tutorial flow
1.6.5	May 22 2025	Add Serbia, Hebrew translations Update UM, SDK, 6L Max connection Add K1000 K6LMax devices support Bug fixes
1.6.6	Jun 27 2025	Internal release, add languages, bug fixes

1.6.7	July 01 2025	Enable K1000 device support, Add 4 more languages translations, bug fixes
1.6.8	Aug 28 2025	Enhancements & UI Updates - Updated Sample App <i>About</i> screen title to “About Kardia Core Android” and added aboutScreenRegion. - Introduced KS Blue Theme and updated image dimensions for KM device UI. Bug Fixes & Maintenance - Removed BLE connection confirmation in 6L and K1000 recording flows. - Unified connection handling using the 12L flow logic. - Fixed crash when switching devices during KS sessions - Added new localization - Implemented UserFlow enum for managing KS flows
1.6.9	Sep 18 2025	UI & Functional Fixes - Fixed text truncation issues on the recording screen. - Updated PDF footer message for clarity. - Fixed crash and minor UI glitches in the tutorial screen. - Improved recording screen UI alignment and text handling. - Updated <i>About Kardia Core Android</i> screen to display Part# instead of Model#. PDF Enhancements - Updated Consumer ECG PDF layout, strings, and footer messages. - Improved text rendering and alignment in PDF generation. Localization & Resources - Fixed Tamil and other translation issues. - Updated localized strings. Bug Fixes & Maintenance - Removed deprecated user-agent string.

1.6.10	Nov 3 2025	Feature Updates & Improvements • Updated BLE layer with improved handlers and K12 simulator support for Kardia System. • Added optional text field in the PDF header page. • Updated TestApp to version 1.6.10-test. • Removed interpretation hazard warning from PDF after ECG confirmation in KardiaPro. • Added API key inclusion in authentication API requests. • Android 16 support Bug Fixes, Maintenance & Cleanup • Corrected sex values displayed as <i>Unknown</i> in PDF reports. • Fixed UI issues on the recording screen for KardiaMobile devices. • Corrected heart rate label display under BPM in recording screen. • Fixed issue where “View Tutorial” button did not respond for KM, KM6L, and K1000 devices. • Fixed issue showing invalid device detected popup. • Improved SDK compatibility with 16KB page sizes on newer Android devices. • Migrated Sample App into ACKit-Android repository and removed hardcoded credentials.
--------	------------	--

OVERVIEW

The AliveCor SDK, paired with the Kardia Mobile or Kardia 6L or Kardia Card device, captures medical grade 30-second electrocardiogram (ECG or EKG) recordings and provides clinically-validated results using FDA-cleared Kardia-AI algorithms.

The AliveCorKit SDK provides a drop-in UI which is a complete, ready-made recording UI offering a quick and easy way to start recording EKG and show FDA-cleared results to customers. UI and data components of this flow are also provided, for SDK clients that wish to provide a customized experience for users.

The AliveCorKit SDK is provided as two AAR libraries: **AliveCorKit** and **AliveCorKitLite**. These libraries provide APIs and UI components that allow for recording EKG, analyzing it via AliveCor's **KardiaAI** framework, and managing recorded EKG data with AliveCor's backend.

Starting from SDK v1.6.10 it supports Android 16 (api 36, build tools 36.0.0), minimum api support is version 26.

AliveCorKitLite

AliveCorKitLite provides functionality for recording and analyzing EKG data.

A simple UI flow for recording EKG is provided along with the flow's component Fragments. SDK clients may use our default recording flow to take users through a complete, FDA-cleared experience. Clients may build their own flows out of the component Fragments to accommodate their needs.

AliveCorKitLite also provides tools for EKG Analysis using the **KardiaAI** toolkit, which is included in the SDK and provides integration with **Kardia PRO** back end.

AliveCorKit

AliveCorKit provides all of **AliveCorKitLite**'s functionality, and also allows clients to authenticate and interact with AliveCor's backend to store EKGs or interact with Kardia Pro practices.

Localization

Currently **AliveCorKit** and **AliveCorKitLite** supports the following languages:

values — English (default)

values-ar — Arabic
values-bg — Bulgarian
values-cs-rCZ — Czech (Czech Republic)
values-cy — Welsh
values-da — Danish
values-de — German
values-el — Greek
values-en-rGB — English (United Kingdom)
values-es — Spanish
values-es-rUS — Spanish (United States)
values-et — Estonian
values-fi — Finnish
values-fr — French
values-he — Hebrew
values-hr — Croatian
values-hu-rHU — Hungarian (Hungary)
values-it — Italian
values-iw — Hebrew (legacy code, same as he)
values-ja — Japanese
values-ko — Korean
values-land — Landscape (layout qualifier, not a language)
values-lt — Lithuanian
values-lv — Latvian
values-ms-rSG — Malay (Singapore)
values-nb — Norwegian Bokmål
values-nl — Dutch
values-pl — Polish
values-pt-rBR — Portuguese (Brazil)
values-pt-rPT — Portuguese (Portugal)
values-ro — Romanian
values-ru — Russian
values-sk — Slovak
values-sl — Slovenian
values-sr — Serbian
values-sv — Swedish
values-ta-rSG — Tamil (Singapore)
values-th — Thai
values-vi — Vietnamese
values-zh-rCN — Chinese (Simplified, China)
values-zh-rTW — Chinese (Traditional, Taiwan)

Get list of SDK supported languages

Starting from SDK 1.6.2 sdk supports different languages for EKG recording, Result Screen and PDF Reports independently from the user phone locale.

How to get supported by SDK language list example:

```
val supportedLanguages: List<LanguageUtils.Language> =  
    AliveCorKitLite.getSupportedLanguages(context)
```

Where each `LanguageUtils.Language` contains `Language` string in english and code, for example

```
Supported Language : (en) English  
Supported Language : (ar) Arabic  
  
Supported Language : (cs) Czech  
Supported Language : (cy) Welsh  
Supported Language : (da) Danish  
Supported Language : (de) German  
Supported Language : (en-rGB) English (UK)  
Supported Language : (es) Spanish  
Supported Language : (fi) Finnish  
Supported Language : (fr) French  
Supported Language : (hu) Hungarian  
Supported Language : (it) Italian  
Supported Language : (ja) Japanese  
Supported Language : (ko) Korean  
Supported Language : (lt) Lithuanian  
Supported Language : (nb) Norwegian Bokmål  
Supported Language : (nl) Dutch  
Supported Language : (pl) Polish  
Supported Language : (pt-rBR) Portuguese (Brazil)  
Supported Language : (pt-rPT) Portuguese (Portugal)  
Supported Language : (ru) Russian  
Supported Language : (sv) Swedish  
.  
.  
.  
Supported Language : (th) Thai  
Supported Language : (vi) Vietnamese  
Supported Language : (zh-rCN) Chinese (Simplified)  
Supported Language : (zh-rTW) Chinese (Traditional)
```

Setup Recording screen language

You can set the SDK UI language any time to call SDK method:

```
// Change the locale to the selected language
```

```
LanguageUtils.setAppLocale(context, selectedLanguage.code,
resources)
```

For more details see the SDK sample app **LanguageSelectionActivity.kt** or **MainActivity.java**

```
@Override
public void onConfigurationChanged(@NonNull Configuration newConfig) {
    super.onConfigurationChanged(newConfig);
    // Apply the selected language when configuration changes
    String selectedLanguageCode =
getIntent().getStringExtra("SELECTED_LANGUAGE_CODE");
    LanguageUtils.setAppLocale(this, selectedLanguageCode, getResources());
}
```

Setup PDF report language

To configure PDF report language new method is available starting from the SDK 1.6.2

```
// new method to select any-supported language in SDK for PDF report
// set to pdfHelper.setLanguage(null); to reset back to default device Locale
if (pdfLanguage != null) {
    pdfHelper.setLanguage(pdfLanguage);
}
```

```
private void generatePdf(String pdfLanguage) {
    // PDF Reports can be generated into any file. The SDK will overwrite any
data in this file
    File pdfFile = new File(mItem.getFilesDirectory(), mItem.getUuid() +
".pdf");

    // PDF Reports can be generated for any fully-populated AliveCorEcg. The
recording files
    // must be present within the directory indicated by AliveCorEcg's
getFilesDirectory()
    // PDFs can be letter-size or A4, depending on preference.
    AliveCorPdfHelper pdfHelper = AliveCorKitLite.get().getPdfHelper();
    pdfHelper.setForceInvert(false);
    // new method to select any-supported language in SDK for PDF report
    // set to pdfHelper.setLanguage(null); to reset back to default device
Locale
    if (pdfLanguage != null) {
        pdfHelper.setLanguage(pdfLanguage);
    }
    pdfHelper.createPdfWithEncryptionPrompt(requireActivity(),
        mItem,
        AliveCorPdfHelper.Size.LETTER,
        pdfFile,
        // Logo and Patient Info are nullable.
    );
}
```

```
    "images/alivecor_logo.png",
    // Each field of PatientInfo is nullable
    new AliveCorPdfHelper.PatientInfo("Jane", "Doe",
        DateTime.now().minusYears(50).getMillis(), "12345", "M"),
    // Tags and Notes are also Nullable
    "Symptom1, Symptom2, Symptom3",
    "Patient presents with some symptoms",
    this, true, false);
}
```

CONFIGURATION AND USAGE

The **AliveCorKit** SDKs are distributed as standalone AAR archive files. **AliveCorKit** and **AliveCorKitLite** are each distributed individually. Once imported, the libraries must have their dependencies satisfied, and require authentication via API key before they can be used.

Import AAR files

One of the **AliveCorKit** or **AliveCorKitLite** libraries must be added to your application project.

- 1) In Android Studio, Go to File -> New Module
- 2) In the next dialog, choose 'Import .JAR/.AAR package'
- 3) Choose the AAR file you wish to use: either "**AliveCorKit-release.aar**" or "**AliveCorKitLite-release.aar**."
- 4) Wait for project sync to finish
- 5) Double check that module build.gradle contains artifacts

```
artifacts.add("default",
file('AliveCorKit-coreRelease-1.6.2.aar'))
```

Or

```
artifacts.add("default",
file('AliveCorKitLite-coreRelease-1.6.2.aar'))
```

Add Dependencies

You must add dependencies to your application module for the AAR module you have imported. The **AliveCorKit** libraries also depend on several additional commonly-used libraries, which you must supply yourself.

- 1) Add the **AliveCorKitLite-release** or **AliveCorKit-release** module to your application's dependencies. Add the following line to the `dependencies` block of your application's `build.gradle`:

```
implementation project(':AliveCorKit-release')
```

Or

```
implementation project(':AliveCorKitLite-release')
```

- 2) Add the additional library dependencies.
 - a) Add the `jcenter` and `google` maven repositories to your project. In your **Project** `build.gradle`, add:

```
allprojects {
    repositories {
        google()
        jcenter()
    }
}

ext {
    buildToolsVersion = '35.0.0'
    buildCompileSdkVersion = 35
    buildTargetSdkVersion = 35
    buildMinSdkVersion = 24

    constraintLayoutVersion = "2.1.4"

    roomVersion = '2.6.1'
    daggerVersion = '2.52'
    daggerHiltVersion = '2.52'
    rxAndroidVersion = '2.1.1'
    rxJavaVersion = '2.2.21'
    gsonVersion = '2.9.0'
    timberVersion = '5.0.1'
    jodaVersion = '2.13.0'
    retrofitVersion = '2.9.0'
    okhttp3 = '4.10.0'

    kardiaAiLibVersion = '2.0.9'
    kardiaAiVersionV2 = '2.0.9'
    kardiaAiVersionV1 = '1.3.4'
```

```
}
```

b) Add the additional dependencies to your **application** build.gradle:

For AliveCorKit

```
dependencies {
    implementation 'com.airbnb.android:lottie:6.5.2'
    implementation "net.danlew.android.joda:${rootProject.ext.jodaVersion}"

    implementation
"io.reactivex.rxjava2:rxandroid:${rootProject.ext.rxAndroidVersion}"
    //rxandroid only updates when something changes in rxandroid. Explicitly
    require semi-current rxjava
    implementation "io.reactivex.rxjava2:rxjava:${rootProject.ext.rxJavaVersion}"

    implementation
"org.jetbrains.kotlin:kotlin-stdlib-jdk8:${rootProject.ext.kotlin_version}"
    implementation "androidx.core:core-ktx:1.13.1"
    implementation "androidx.appcompat:appcompat:1.7.0"
    implementation "com.google.android.material:material:1.12.0"
    implementation "androidx.constraintlayout:constraintlayout:2.1.4"
    implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.8.6"
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.6"
    implementation "androidx.navigation:navigation-fragment-ktx:2.8.1"
    implementation "androidx.navigation:navigation-ui-ktx:2.8.1"
    implementation "com.jakewharton.timber:timber:5.0.1"
    implementation "androidx.datastore:datastore-preferences:1.0.0"
    implementation "com.google.android.gms:play-services-location:20.0.0"

    implementation 'androidx.arch.core:core-runtime:2.2.0'
    implementation 'androidx.lifecycle:lifecycle-livedata-core:2.8.6'
    implementation 'androidx.lifecycle:lifecycle-viewmodel:2.8.6'
    implementation 'androidx.activity:activity:1.9.2'
    implementation 'androidx.lifecycle:lifecycle-runtime:2.8.6'
    implementation 'androidx.lifecycle:lifecycle-common:2.8.6'
    implementation 'androidx.fragment:fragment:1.8.3'

    implementation
"com.squareup.retrofit2:retrofit:${rootProject.ext.retrofitVersion}"
    implementation
"com.squareup.retrofit2:converter-gson:${rootProject.ext.retrofitVersion}"
    implementation
"com.squareup.retrofit2:adapter-rxjava2:${rootProject.ext.retrofitVersion}"
    implementation
"com.squareup.retrofit2:converter-scalars:${rootProject.ext.retrofitVersion}"

    implementation "com.jakewharton.timber:timber:${rootProject.ext.timberVersion}"

    implementation "androidx.room:room-runtime:${rootProject.ext.roomVersion}"
    kapt "androidx.room:room-compiler:${rootProject.ext.roomVersion}"

    //dependency injection
    implementation "com.google.dagger:dagger:${rootProject.ext.daggerVersion}"
}
```

```

        kapt "com.google.dagger:dagger-compiler:${rootProject.ext.daggerVersion}"
        implementation
    "com.google.dagger:dagger-android:${rootProject.ext.daggerVersion}"
        compileOnly
    "com.google.dagger:dagger-android-support:${rootProject.ext.daggerVersion}"
        kapt
    "com.google.dagger:dagger-android-processor:${rootProject.ext.daggerVersion}"

}

```

For AliveCorKitLite

```

dependencies {

    implementation
    "org.jetbrains.kotlin:kotlin-stdlib-jdk8:${rootProject.ext.kotlin_version}"
    implementation "androidx.core:core-ktx:1.13.1"
    implementation "androidx.appcompat:appcompat:1.7.0"
    implementation "com.google.android.material:material:1.12.0"
    implementation "androidx.constraintlayout:constraintlayout:2.1.4"
    implementation "androidx.lifecycle:lifecycle-livedata-ktx:2.8.6"
    implementation "androidx.lifecycle:lifecycle-viewmodel-ktx:2.8.6"
    implementation "androidx.navigation:navigation-fragment-ktx:2.8.1"
    implementation "androidx.navigation:navigation-ui-ktx:2.8.1"
    implementation "com.jakewharton.timber:timber:5.0.1"
    implementation "androidx.datastore:datastore-preferences:1.0.0"
    implementation "com.google.android.gms:play-services-location:20.0.0"

    implementation "net.danlew:android.joda:${rootProject.ext.jodaVersion}"
    implementation
    "io.reactivex.rxjava2:rxandroid:${rootProject.ext.rxAndroidVersion}"
    //rxandroid only updates when something changes in rxandroid. Explicitly
    require semi-current rxjava
    implementation "io.reactivex.rxjava2:rxjava:${rootProject.ext.rxJavaVersion}"
    implementation "com.jakewharton.timber:timber:${rootProject.ext.timberVersion}"
    implementation 'androidx.appcompat:appcompat:1.7.0'

    def fragment_version = "1.5.0"
    // Java language implementation
    implementation "androidx.fragment:fragment:$fragment_version"
    // Kotlin
    implementation "androidx.fragment:fragment-ktx:$fragment_version"
    // Testing Fragments in Isolation
    debugImplementation "androidx.fragment:fragment-testing:$fragment_version"

    //dependency injection
    implementation "com.google.dagger:dagger:${rootProject.ext.daggerVersion}"
    kapt "com.google.dagger:dagger-compiler:${rootProject.ext.daggerVersion}"
    implementation
    "com.google.dagger:dagger-android:${rootProject.ext.daggerVersion}"
        compileOnly
    "com.google.dagger:dagger-android-support:${rootProject.ext.daggerVersion}"

}

```

Note: one more important dependency was recently added

```
implementation 'androidx.security:security-crypto:1.1.0-alpha03'
```

Note: that `com.jakewharton:butterknife` dependency was removed in SDK 1.4.0 as it's conflicting with Android 12 API, please use native api instead

```
buildFeatures {  
    viewBinding = true  
    dataBinding = true  
    buildConfig = true  
}
```

Configure ProGuard

In order to make release builds including **AliveCorKit** or **AliveCorKitLite** Proguard must be configured to keep AliveCor symbols. The configuration is simply:

```
# Keep all AliveCor symbols  
-keep class com.alivecor.** {*;}
```

Initialize the SDK Modules

In order to use the **AliveCorKit** SDK, you must supply it first with a json web token obtained through your backend. Documentation for this can be found with the backend module. Once your token is obtained, you must supply it to the `initialize()` method for your chosen SDK.

If you do not initialize the libraries, you will be unable to use them.

You should initialize them once every time you want to do a recording.

For example (taken from the Sample App), **AliveCorKitLite** can be initialized like this :

```
private void prepareToRecord(Runnable onInit) {  
  
    ProgressDialog dialog = new ProgressDialog(this);  
    dialog.show();  
  
    TokenRequest tokenRequest = new TokenRequest();  
}
```



```

tokenRequest.setPartnerId("PKtxN7pRrQW1blMCcH4Ac66ta5rflozk");
tokenRequest.setBundleId("bid1");
tokenRequest.setPatientMrn("Patient1");
tokenRequest.setTeamId("Fs0ZK1raZi5KidNtCyK9b4nyh6gkq4ah");
tokenRequest.setTeamId("YourTeamID");

// where authToken is Base64 your backend login and password
String authToken = "Basic eHozMmdobXc6cHdkZjNqNw=";
Call<TokenResponse> call = new RestClient().getApi().token(authToken,
tokenRequest);
call.enqueue(new Callback<TokenResponse>() {
    @Override
    public void onResponse(Call<TokenResponse> call,
Response<TokenResponse> response) {
        Timber.d("MainActivity Response is %s", response.body());
        if (response.body() != null) {
            AliveCorKitLite.initialize(MainActivity.this,
                response.body().getJwt(),
                new InitListener() {
                    @Override
                    public void onInitComplete() {
                        Timber.d("AliveCorKitLite initialized
version", AliveCorKitLite.getVersion());
                        mAliveCorKitLiteInitialized = true;
                        dialog.dismiss();
                        onInit.run();
                    }

                    @Override
                    public void onInitError(Throwable throwable) {
                        dialog.dismiss();
                        Timber.d("SDK Version is %s",
AliveCorKitLite.getVersion());
                        Timber.e("Error initializing
AliveCorKitLite", throwable);
                    }
                }, AliveCorServer.STAGING_US, "MyApplication",
                tokenRequest.getBundleId(),
                tokenRequest.getPartnerId(),
                true);
        } else {
            View view = findViewById(R.id.mainLayout);
            Snackbar.make(view, "Get jwt token call failed",
Snackbar.LENGTH_LONG).show();
            dialog.dismiss();
        }
    }

    @Override
    public void onFailure(Call<TokenResponse> call, Throwable t) {
        dialog.dismiss();
        Timber.e("MainActivity", "Error is ", t);
    }
}

```

```
});  
}
```

Make sure that you are using appropriate tokenRequest Ids for STAGING or PROD env.

... and **AliveCorKit** can be initialized like this :

```
// Example in Kotlin  
// Before AliveCorKit.initialize you will need to get JWT  
  
AliveCorKit.initialize(context, jwt,  
    object : InitListener {  
        override fun onInitComplete() {  
            Timber.i("AliveCorKit init success")  
            onSuccess()  
        }  
  
        override fun onInitError(t: Throwable?) {  
            Timber.e(t, "AliveCorKit init error")  
            onFailure(t)  
        }  
    },  
    App_name, // application name  
    getUserAgent(),  
    Null, // partner Id  
    server, // AliveCorServer  
    False // enableConsoleLogging  
)  
  
// Where userAgent is a combination of strings  
fun getUserAgent(): String {  
    return java.lang.String.format(  
        USER_AGENT_FORMAT,  
        getString(R.string.app_name),  
        BuildConfig.VERSION_NAME,  
        Build.VERSION.RELEASE,  
        BuildConfig.APPLICATION_ID,  
        Build.MODEL  
    )  
}
```

Feature Access and Expiration

Note that different SDK Users may have access to different supported devices. Not all Kardia devices may be allowed for use with your particular account.

The recording screens will throw `Exceptions` if a configured device is not allowed for your key.

Methods are provided in the `AliveCorKit` and `AliveCorKitLite` classes for determining which devices are supported.

For example:

```
List<AliveCorDevice> supportedDevices =  
    AliveCorKitLite.get().getSupportedDevices();  
// Additional Logic
```

Launch the Default Recording Flow

The **AliveCorKit** libraries provide an `Activity` that can do an entire Recording Flow, from potentially pairing a KardiaMobile 6L device over Bluetooth, collecting EKG data, interpreting the result, and displaying it to the user. The UI and data functionality in the default flow is considered reliable and has been cleared by the FDA.

The default flow is provided as an **Activity** that can optionally return **Result** information. The activity does all required data handling during the flow, depending on the scope of the library used.

AliveCorKitLite

For **AliveCorKitLite**, the `Intent` which starts the flow can be obtained from the initialized **AliveCorKitLite** object. Result data is provided as a `Parcelable` which can be parsed into a readable format using provided methods. For example (relevant parts bolded for emphasis):

```
public class MainActivity extends AppCompatActivity {  
    private static final int REQUEST_RECORD_EKG = 101;  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
    }  
  
    @OnClick(R.id.start_recording)  
    public void startRecordingClicked() {  
        initAliveCorKitAsync(() -> startRecording()) // Example above  
    }  
}
```

```

public void startRecording() {
    Intent intent = AliveCorKitLite.get()
        .getRecordIntent(AliveCorDevice.TRIANGLE);
    //intent.putExtra("com.alivecor.ecg.record.HELP_URLS",
createHelpUrls()); // Example to add overwrite Help Url for default
flow, where you need to create RecordingHelpUrls object.
    startActivityForResult(intent, REQUEST_RECORD_EKG);
}

@Override
protected void onActivityResult(int requestCode, int resultCode,
@Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if(requestCode == REQUEST_RECORD_EKG) {
        Log.i(TAG, String.format("Back from EKG Recording. Result:
%d/%s",
            resultCode, AliveCorKitLite.get()
                .getRecordActivityResult(data)));
    }
}
}

```

Additional information about valid inputs and output format can be found in the reference.

Create a Custom Recording Flow with Premade UI Components

If the Default Recording Flow does not suit your needs, both **AliveCorKit** and **AliveCorKitLite** provide the Default Flow's individual Fragments, **RecordEkgFragment** and **ResultEkgFragment**, for use by SDK clients. Additionally, both libraries provide **EcgResultView** to SDK clients, a lighter-weight alternative to **ResultEkgFragment** that can show an ECG trace, a KardiaAI Finding (ie, Normal, Afib, Too Short, etc), and allow toggling inversion (for single-lead EKGs).

RecordEkgFragment performs the recording, prompting for user intervention to resolve permission, configuration, or recording issues. It calls back when these problems occur, but no intervention is required from the SDK client to resolve them. When a recording has been completed, it will call back with an **AliveCorEcg** object representing the completed recording. The **RecordEkgFragment** communicates with its hosting Activity (or parent Fragment) using the **RecordEkgListener**.

ResultEkgFragment displays the results of a single recording, presenting the finding, the ECG trace, BPM, and a summary of the results. Optionally, this Fragment can present an app bar, in order to be used as a full-screen experience. **ResultEkgFragment** communicates with its hosting Activity or Parent Fragment using **ResultEkgListener**.

EcgResultView also displays the results of a single recording, though it does not present any explanation text, help links, etc. It simply presents the Finding, BPM, and Trace of a completed EKG, along with a simple control for inverting the ECG. Its interface is much simpler than **ResultEkgFragment**, requiring only the **AliveCorEcg** that should be displayed. The **EcgResultView** also shows additional information regarding the possible findings when the user clicks on an icon it contains. If you wish to customize the information displayed (or disable the button) you can use **EcgResultView.CpomInfoListener** to override the default behavior.

Note that neither Fragment stores these recorded ECGs to a persistent datastore. Files containing the recorded samples will be saved to disk at the paths indicated.

But any additional storage (such as saving the results in a searchable database or uploading them to a remote server) must be provided by the client app.

An implementation of a custom flow using both Fragments and storing the results can be found in the sample app, in **CustomFlowActivity**. The following example snippets are adapted from the Sample App.

Starting **RecordEkgFragment**:

```
// ... Having already waited for init to finish with your jwt

try {
    RecordingConfiguration configuration =
        new RecordingConfiguration();
    configuration.setDevice(device);
    configuration.setLeads(LeadConfiguration.SINGLE);
    configuration.setFilterType(FilterType.ENHANCED);
    configuration.setMaxDurationSeconds(30);
    configuration.setResetDurationSeconds(10);
    // You may need to set this to 50 depending on your location
    configuration.setMainsFrequency(
        RecordingConfiguration.MAINS_FREQUENCY_60Hz);

    RecordEkgFragment fragment =
        RecordEkgFragment.newInstance(configuration);
    // This listener should be removed when the Activity shuts
down
    fragment.addListener(...);
    getSupportFragmentManager()
```

```

        .beginTransaction()
        .replace(frag, R.id.test_frag_container)
        .commit();
    } catch (InvalidArgumentException e) {
        throw new RuntimeException(e);
    }
    //...

```

Starting the ResultEkgFragment:

```

// Listener for completed recordings from RecordEkgFragment
private RecordEkgListener createRecordEkgListener() {
    return new RecordEkgListener {
        // ...
        @Override
        public void onRecordCompleted(AliveCorEcg result) {
            replaceResultFragment(result);
            // The ECG record could be saved to disk at this point, if
            // desired.
        }
        // ...
    }
}

// Creates and adds a new ResultEkgFragment
private void replaceResultFragment(@NonNull AliveCorEcg result) {
    ResultEkgFragment fragment =
        ResultEkgFragment.newInstance(result);

    // The ResultEkgFragment will call back when the user signals
    that
    // they are done with this ECG. If you are not using the
    built-in
    // Toolbar, you will
    // not receive this callback and must handle this case in some
    // other way
    fragment.addListener(...);
    getSupportFragmentManager().beginTransaction()
        .replace(R.id.custom_flow_fragment_container, fragment)
        .commit();
}

```

Using EcgResultView

```
// ...
// Obtain an AliveCorEcg from somewhere (eg, RecordEkgFragment or a
// Repository)
AliveCorEcg ecg = getRecordedEcg();

EcgResultView resultView = findViewById(...);
resultView.setEcgRecord(ecg);

// The Result View contains an information icon which can be
// clicked
// to provide additional information about a result
resultView.setCpomInfoListener(() -> showInfoDialog());

// ...
```

Display the Trace for a Recorded ECG

Both **AliveCorKit** and **AliveCorKitLite** offer the class **RhythmStripView**. This View allows you to display recorded ECG data on an optional grid, retaining the correct scale between both trace and grid. **RhythmStripView** can be styled and configured in a number of ways, depending on your needs.

For a **Full View** of the ECG, which can be scrolled horizontally to view the entire strip for example RecordingDetailsFragment from the sample app has this functionality:

```
// ...
<!-- Display as a Full View with horizontal scrolling, and
A darker-gray Grid -->
<com.alivecor.view.RhythmStripView
    android:id="@+id/recording_detail_strip"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    app:displayMode="full"
    app:majorColorGrid="@android:color/darker_gray"
    app:minorColorGrid="@android:color/darker_gray"
    app:traceStrokeColor="@color/black"
/>
//...
```

The **Full View** can also be used with a set height, if you don't want it to fill the entire container.

For a **Preview** of the ECG, which shows a piece near the center of the trace in a non-scrollable way:

```
// ...
<!-- Display as a Preview with no scrolling, and with the grid
      Lines gone (by setting them to the same color as the bg) -->
<com.alivecor.view.RhythmStripView
    android:id="@+id/recording_list_content_strip"
    android:layout_width="match_parent"
    android:layout_height="250dp"
    android:background="@color/white"
    app:displayMode="preview"
    app:majorColorGrid="@color/white"
    app:minorColorGrid="@color/white"
    app:traceStrokeColor="@color/black"
    app:traceStrokeWidth="2dp" />
// ...
```

Generate PDF Reports for Recorded ECGs

Both **AliveCorKit** and **AliveCorKitLite** allow PDF reports to be generated for recorded EKGs. The report includes some basic information about the user, as well as a printout of the ECG trace in front of a grid for scale. The **AliveCorPdfHelper** class can be obtained from either **AliveCorKit** or **AliveCorKitLite**.

```
// ...
AliveCorEcg aliveCorEcg = ...; // Get from your persistent db

// PDF Reports can be generated into any file.
// The SDK will overwrite any data in this file
File pdfFile =
    new File(mItem.GetFilesDirectory(), mItem.getUuid() + ".pdf");

// PDF Reports can be generated for any fully-populated
// AliveCorEcg. The recording files
// must be present within the directory indicated by
AliveCorEcg's
// getFilesDirectory()
// PDFs can be letter-size or A4, depending on preference.
// PatientInfo is mandatory data to fill user info on the report
// algorithmVersion is a Kardia AI string version "KAIV1",
"KAIV2"
// and new arrays beatsData, avgBeatData, rrIntervalData for
// additional info in the reports, that could be pass as null
// for "KAIV1".
AliveCorKitLite.get().getPdfHelper().createPdfReport(aliveCorEcg,
    AliveCorPdfHelper.Size.LETTER,
    pdfFile, "images/alivecor_logo.png",
    new AliveCorPdfHelper.PatientInfo("Jane", "Doe",
    DateTime.now().minusYears(50).getMillis(), 12345"),
    tagsString, notesString, algorithmVersion,
    beatsData, avgBeatData, rrIntervalData,
    enablePvcNotation, isEcgRecordedByUser);

//...
```

Set PDF report language (pdfLanguage is one of 26 the SDK supported language codes, "en", "it", "fr" etc.). To reset it back to the device locale set it to `null`.

```
// new method to select any-supported language in SDK for PDF report
// set to pdfHelper.setLanguage(null); to reset back to default device Locale
if (pdfLanguage != null) {

    pdfHelper.setLanguage(pdfLanguage);

}
```

Use the Encryption Password Prompt

For many cases, protection of user PHI may require the PDF report to be encrypted. While the library does not provide encryption itself, it *does* provide a UI for prompting for a password for encryption. Use of this prompt may be required by regulation depending on your exact business situation. The **AliveCorPdfHelper** class offers a quick way to show a valid and acceptable prompt for this purpose, with hooks allowing for you to add your own secure encryption logic.

This method also allows (technically) asynchronous PDF creation. While the report is still generated on the main thread, success and error information will be reported via Listener.

An example can be found here, also replicated in the Sample app's **RecordingDetailFragment**.

```
// In a class implementing AliveCorPdfHelper::PdfListener

/**
 * @param forceEncryption In some regulatory domains, it may be
 * required to encrypt all PDF reports.
 * note: this method is for For backward compatibility
 * where this should implement and overwrite PdfListener interface
 * to receive PDF report callbacks when it's ready
 */
private void generatePdf(boolean forceEncryption) {
    // PDF Reports can be generated into any file. The SDK will
    // overwrite any data in this file
    File pdfFile = new File(mItem.getFilesDirectory(),
        mItem.getUuid() + ".pdf");
    AliveCorKitLite.get().getPdfHelper()
        .createPdfWithEncryptionPrompt(
            requireActivity(), mItem,
            AliveCorPdfHelper.Size.LETTER,
            pdfFile, "images/alivecor_logo.png",
            new AliveCorPdfHelper.PatientInfo("Jane", "Doe",
                DateTime.now().minusYears(50).getMillis(), "12345"),
            tagsString, notesString
        );
}
```

```

        this, true, forceEncryption);
    }

    @Override
    public void onEncryptPdf(@NonNull File file, @NonNull String s) {
        // Fill in your own encryption logic here. The SDK does not
        // contain any code allowing PDF Encryption
    }

    @Override
    public void onShowPdf(@NonNull File file) {
        showPdf(file);
    }

    @Override
    public void onError() {
        // Report Errors
    }
}

```

Starting from SDK 1.4.0 is recommended to use new PDF report methods with additional parameters to support Kardia V2 information.

1. createPdfReport

```

public boolean createPdfReport(@NonNull AliveCorEcg ecg,
    @NonNull Size size, @Nullable File outputFile,
    @Nullable String logoAsset, PatientInfo patientInfo,
    @Nullable String tags, @Nullable String notes,
    String algorithmVersion,
    List<List<Integer>> beatsData,
    List<Short> avgBeatData,
    List<List<Integer>> rrIntervalData,
    boolean enablePvcNotation,
    boolean isEcgRecordedByUser)

```

2. createPdfWithEncryptionPrompt

```

public void createPdfWithEncryptionPrompt(@NonNull Activity activity,
    @NonNull AliveCorEcg ecg,
    @NonNull Size size, @NonNull File outputFile,
    @Nullable String logoAsset, PatientInfo patientInfo,
    @Nullable String tags, @Nullable String notes,
    PdfListener pdfListener, boolean showEncryptionPrompt,
    boolean forceEncryption,
    String algorithmVersion,
    List<List<Integer>> beatsData,
    List<Short> avgBeatData,
    List<List<Integer>> rrIntervalData,

```

```
boolean enablePvcNotation,  
boolean isEcgRecordedByUser)
```

class AliveCorPdfHelper

Represents metadata associated with an AliveCor EKG Recording

Properties / Methods	Description
Boolean mForceInvert	mForceInvert Boolean flag can have one of three states: • null - default state, SDK will work as before and automatically take evaluation.isInverted() flag from AI evaluation • true - forcibly invert EKG in the PDF, and will ignore evaluation.isInverted() internal AI evaluation • false - forcibly ignore evaluation.isInverted() flag and always will draw EKG as it was taken originally
<i>public void setForceInvert(boolean forceInvert);</i>	Use this method to set the mForceInvert flag for AliveCorPdfHelper before creating a PDF report.

Analyze Data Recorded from Other Sources

Both **AliveCorKit** and **AliveCorKitLite** can analyze pre-recorded ECG data. To analyze recorded samples, use the **ClassifyRunner** class. The **classifySamples()** method takes double-precision, millivolt-resolution ECG data and returns an evaluation of the ECG.

To analyze captured ECG data directly (assuming a typical 300 Hz signal, recorded with a 50Hz Mains Filter by default):

```
// ...  
// Client provides the samples from their own data source  
int samplesLen = calculateSamples();
```

```
double[] samplesMv = loadSamplesFromFile();

ClassifyRunner runner = AliveCorKitLite.get().getClassifyRunner();
ClassifyRunner.RunResult result = runner.classifySamples(samplesMv,
samplesMv.length, MainsFilterFrequency.MAINS_50_HZ.ordinal(),
SampleRate.SAMPLE_RATE_300_HZ);

Log.d(TAG, "Result is " + result);
//...
```

Login and Manage ECG Data with AliveCorKit

The **AliveCorKit** full library includes tools for storing recorded ECG data, as well as authenticating and interacting with **AliveCor's Kardia Pro backend servers**. These features can be accessed using the `KardiaProEcgBackend` class, which is provided by `AliveCorKit.getEcgBackend()`

Login/Logout with a Kardia Pro Account

In order to interact with AliveCor's backend servers, you must authenticate with a valid Kardia Pro username and password. Once authenticated, the Kardia Pro user's basic Profile and Team Member data will be stored by the SDK for later use. Login sessions persist between application shutdowns, and can be created and terminated with methods in `KardiaProEcgBackend`.

No UI is supplied for authentication. Within your login UI, you must call `KardiaProEcgBackend::loginWith()`. For example (important code bolded),

```
// LoginFragment.java
// ...
private TextView emailInput;
private TextView passwordInput;

private KardiaProEcgBackend aliveCorBackend;
private Disposable loginSubscription;

@Override
public void onCreate(...) {
    // ...
    aliveCorBackend = AliveCorKit.get().getEcgBackend();
    // ...
}

@Override
public View onCreateView(...) {
```

```

// ...
emailInput = contentView.findViewById(...);
passwordInput = contentView.findViewById(...);
// ...
}

@OnClick(R.id.loginButton)
private void loginClicked(View v) {
    String username = emailInput.getText();
    String password = passwordInput.getText();

    loginSubscription =
        aliveCorBackend.loginWith(username, password)
            .observeOn(AndroidSchedulers.mainThread())
            .subscribe(this::loginSuccess,
this::loginError);
}

private void loginSuccess(String authToken) {
    Log.v("Login Success! Logged-in User Data: " +
backend.getOwnTeamMember());
    // Handle login success in UI
}

//...

```

To Log Out the current Team Member, simply subscribe to `KardiaProEcgBacked::logout`:

```

@Override
public View onCreateView(...) {
    // ...
    logoutButton = contentView.findViewById(...);
    // ...
}

@OnClick(R.id.logoutButton)
private void loginClicked(View v) {
    String username = emailInput.getText();
    String password = passwordInput.getText();

    loginSubscription =
        aliveCorBackend.logout()
            .observeOn(AndroidSchedulers.io())
            .subscribe(this::logoutSuccess, this::logoutError);
}

private void logoutSuccess() {
    Log.v("Logout Success!");
}

```

```

    assertThat(aliveCorBackend.getOwnTeamMember() == null);
    // Handle logout in UI
}

//...

```

Interact with the Logged-In Team Member Profile

Once authenticated, the user's profile can be accessed through `KardiaProEcgBackend::getOwnTeamMember`. The returned object will contain the Kardia Pro team member's profile, as well as information about all the Teams they are a part of.

```

//...
TeamMember loggedInTeamMember =
    AliveCorKit.get().getEcgBackend().getOwnTeamMember();

// Print basic profile information
Log.v(TAG, loggedInTeamMember.getProfile().toString());
// Print the Team Member's Profile ID
Log.v(TAG, loggedInTeamMember.getProfile().getId());

// Print all teams associated with this Team Member
Log.v(TAG, loggedInTeamMember.getTeams());
//...

```

Validate Patient IDs for a Team

Patients within KardiaPro are associated with a generic Patient ID. A patient ID is a string of characters that identifies a patient within a practice. To accommodate different team workflows, the SDK assumes developers have patient IDs already determined externally. Patient IDs for a given Team must conform to a regex pattern, supplied in the Team configuration.

```

//...

// Patient ID input from the UI, etc
String patientId = ...;
TeamMember loggedInTeamMember =
    AliveCorKit.get().getEcgBackend().getOwnTeamMember();

// Validate the Patient ID
if (AliveCorKit.get().getEcgBackend().validatePatientId(patientId)) {
    Intent go = AliveCorKit.get()
        .getRecordIntent(AliveCorDevice.TRIANGLE,
patientId);
    startActivityForResult(REQUEST_CODE_RECORD, go);
}

```

```
} else {  
    showErrorDialog(...);  
}  
  
//...
```

Store and Upload ECG Recordings

The **AliveCorKit** full library provides the class `KardiaProEcgRepository` for storing and reading recorded ECG data. The Repository provides access to both a local database and the AliveCor backend servers. The Repository operates on `KardiaEcgEntity` objects, which can be obtained either by reading them from the database, or mapping them from `RecordingResult` objects using `AliveCorBackendUtil`. Examples are below, and more information about the Repository and its model objects can be found in the Reference.

If you use the Default UI Flow for **AliveCorKit**, (via `AliveCorKit.getRecordIntent()`) then data will be saved and handled automatically. Examples in this section only apply to custom recording UI flows, or clients with EKG-data-related features.

Note: The Repository currently does not support downloading ECG Records related to the team, only upload.

Load ECG Records from Storage

ECG recordings can be read locally using the methods `getEcgRecord`, `getAllLocalEcgRecords`, etc. These methods operate on `KardiaEcgEntity` objects, whose properties are documented in the reference.

For instance, to read all ECG records from the database, you could simply do:

```
//...  
  
AliveCorKit.get().getEcgBackend().getRepository()  
    .getAllLocalEcgRecords()  
    .doOnNext(rec -> Log.d(TAG, "Read Record " + rec))  
    .observeOn(...)  
    .subscribe(...);  
  
//...
```

Save a New ECG Recording

If you are not using the default recording flow, it will be your responsibility to save the recorded ECGs to the database yourself. The Record is available to be saved as soon as `RecordEkgFragment` has called `onRecordResult` on its listener.

`RecordEkgFragment` outputs objects of type `RecordResult`, which can be mapped into a `KardiaEcgEntity` using provided tools. For example:

```
//...

// gotten via AliveCorKit.get().getBackend()
KardiaProEcgBackend backend;
// gotten from user input, or chosen from TeamMember.getTeams()
String teamId;
// gotten from user input
String patientId;

void onRecordResult(RecordResult result) {
    KardiaEcgEntity toSave
        = AliveCorKitUtil.entityFromRecordResult(result);
    dbEntity.getMetadata()
        .put(AliveCorKitUtil.ECG_METADATA_PATIENT_MRN, patientId);
    dbEntity.getMetadata()
        .put(AliveCorKitUtil.ECG_METADATA_TEAM_ID, teamId);

    backend.getRepository().writeEcgRecord(toSave)
        .observeOn(...)
        .subscribe(...);
}

//...
```

Upload ECG Recordings to AliveCor Servers

`KardiaProEcgRepository` can upload ECG Recordings to AliveCor's backend servers, if a Team Member is logged in. The Record must have the `AliveCorKitUtil.ECG_METADATA_PATIENT_MRN` and `AliveCorKitUtil.ECG_METADATA_TEAM_ID` fields set in its metadata in order for upload to succeed.

For example, instead of simply writing the record to disk, as in the above example, we could upload it. Uploaded ECG Records will be saved to the disk. If there was an error during upload, the upload may be attempted again in the future.

```
//...

// gotten via AliveCorKit.get().getBackend()
KardiaProEcgBackend backend;
// gotten from user input, or chosen from TeamMember.getTeams()
String teamId;
// gotten from user input
String patientId;

void onRecordResult(RecordResult result) {
    KardiaEcgEntity toSave
        = AliveCorKitUtil.entityFromRecordResult(result);
    dbEntity.getMetadata()
        .put(AliveCorKitUtil.ECG_METADATA_PATIENT_MRN, patientId);
    dbEntity.getMetadata()
        .put(AliveCorKitUtil.ECG_METADATA_TEAM_ID, teamId);

    backend.getRepository().uploadEcgRecord(toSave)
        .observeOn(...)
        .subscribe(...);
}

//...
```

“Syncing” is also possible via this Repository. The Repository “syncs” by uploading all un-uploaded ECG records. For example:

```
//...

// gotten via AliveCorKit.get().getBackend()
KardiaProEcgBackend backend;

@Override
public void onCreate(...) {
    // ...

    // Upload all un-uploaded ECG Records
    backend.getRepository().uploadDirtyEcgs().subscribe(...);
}

//...
```

API REFERENCE

AliveCorKitLite Classes

com.alivecor.api

This package contains the `AliveCorKitLite` class for configuring the SDK and accessing its default functions.

class `AliveCorKitLite`

Manages configuration for the `AliveCorKitLite` library, providing information about the library as well as helpers for doing the default EKG Recording Flow

Public Methods

```
public static AliveCorKitLite initialize(Context context, String apiKey, InitListener initListener, String appName, boolean enableConsoleLogging)
```

Initializes `AliveCorKitLike`. Call this early in your application's startup, preferably in an `Application` subclass.

Parameters:

context: A `Context` in which `AliveCorKitLite` will run. This `Context`'s `applicationContext` will be used.

appName: The name of your app. This will appear in the copy of certain strings in the premade UIs

apiKey: A valid `AliveCor` API Key. Please contact `AliveCor` sales for a valid API Key.

initListener: A `Listener` for after the asynchronous init has completed

enableConsoleLogging: If true, the `AliveCorKitLite` library will output to logcat as it operates. Note that If you are using the `Timber` logging library, this method will add a `Tree` for logging if this parameter is true. If you install your own trees, you will probably see our log messages.

```
public static AliveCorKitLite get()
```

Gets an instance of `AliveCorKitLite`. Do not call before calling `initialize()`

```
public Intent getRecordIntent(@NonNull AliveCorDevice device)
```

Get an Intent that can launch the Default Flow. for the given device. To see what devices are supported, use `getSupportedDevices()`. Information about the result of the recording will be delivered by `onActivityResult()`, assuming the Activity is launched with `startActivityForResult()`.

The Record Activity can return the following result data:

Result Codes

`RESULT_OK` is returned if the UI flow ended with the user making and accepting a recording

`RecordEkgConstants.RESULT_CHANGE_DEVICE` is returned if the user indicates they wish to switch devices. Calling Activities should launch a UI to choose a different device, and relaunch with this new device.

`RESULT_USER_CANCEL` is returned if the user did not successfully make a recording during their time in the UI flow

Result Data

In most cases, the result Intent will contain data that can be unmarshalled using the `getRecordActivityResult()` method of this class. The `RecordActivityResult` will contain an object representing the successful recording, as well as any non-fatal errors encountered during the flow. These errors should not be considered evidence of failure. If the recording was successful. The result code will be `RESULT_OK` and `RecordActivityResult#isSuccessful()` will return `true`.

```
@Nullable public RecordActivityResult getRecordActivityResult(@Nullable Intent data)
```

Parses the result of the Default Record Activity (obtained from `onActivityResult()`) and returns it. If the result intent did not contain any result data this method will return `null`

```
public EkgAnalyzer getClassifier()
```

Get a **EkgAnalyzer** that can be used to analyze recorded ECG data without a UI.

```
public static String getVersion()
```

Returns the version of `AliveCorKitLite`

```
public List<AliveCorDevice> getSupportedDevices()
```

Returns a list of all devices supported by the library, once initialized with a valid API key.
Attempts to begin recording with an unsupported device will result in Exceptions being thrown

```
public static DialogFragment showAboutFragment(FragmentActivity context);
```

Create and show the SDK About dialog, Parent `FragmentActivity` is required to launch the screen.



Classes Shared Between AliveCorKit and AliveCorKitLite

com.alivecor.api

enum AliveCorDevice

This enum represents the devices supported by AliveCorKit and AliveCorKitLite

Values

KARDIA_CARD

Represents the KardiaMobile Card EKG Device

KARDIA_MOBILE

Represents the Kardia Mobile EKG Device

KARDIA_BAND

Represents the Kardia Band ECG watchband. **The Android SDK does not support this device**

TRIANGLE

Represents the KardiaMobile 6L bluetooth six-lead device

OMRON_COMPLETE

Represents the Omron Complete Blood Pressure and EKG device

UNSPECIFIED

The device is not specified. This value is not generally an acceptable argument to methods like `AliveCorKit#getRecordIntent()`, etc.

class EkgAnalyzer

Analyzes Recorded ECG Data either from recorded samples, or from an ATC file.

Methods

```
public Result classifySamples(double[] samplesMv, int samplesLen, int mainsFrequency,
    SampleRate sampleRate) {
```

Analyzes the given ECG Samples and returns a result based on the analysis. Depending on hardware, this operation may be slow to complete. The input should contain the list of recorded samples for a particular lead of a given recording. This method **does not** provide a final analysis for a multiple-lead ECG recording. The samples are expected to be in units of Millivolts.

Parameters

samplesMv: The recorded ECG samples, in millivolts, to analyze.

samplesLen: The number of samples to consider from the input data, starting at index 0

sampleRate: The Sample Rate of the recorded ECG Data

mainsFrequency: The mains-filter frequency that should be applied to the input data, use

```
MainsFilterFrequency.MAINS_50_HZ.ordinal()
```

or

```
MainsFilterFrequency.MAIN_60_HZ.ordinal()
```

class EkgAnalyzer.Result

Represents the result of a Classification operation.

Properties:	
AliveCorDetermination determination	The final AliveCorDetermination from KardialAI
int hr	The analyzed heart-rate of the recording, if it could be determined
String errorMsg	If classification failed, a message related to the failure
String kardiaAiVersion	The version of KardialAI used to analyze the samples

class AliveCorEcg

Represents metadata associated with an AliveCor EKG Recording

Properties

```
long durationMs
```

The duration, in milliseconds, of the recording.

```
String rawAtcPath
```

The path to this ECG's Raw (unfiltered) ATC file

```
String enhancedAtcPath
```

The path to this ECG's Enhanced-Filtered ATC file

File **filesDirectory**

The directory where the ATC files are stored

long **recordedAtMs**

The time, in UTC milliseconds, that the ECG was recorded

String **uuid**

A unique UUID belonging to the recorded ECG.

RecordingDeviceInfo **deviceInfo**

Contains further information about the device that recorded this ECG

RecordingConfiguration **recordingConfiguration**

Contains the RecordingConfiguration for this ECG

EcgEvaluation **ecgEvaluation**

Contains information about the KardiaAI evaluation performed on this recording

class RecordingDeviceInfo

Contains data about the device that recorded a particular ECG.

Properties

AliveCorDevice **aliveCorDevice**

The type of device that made this recording

String **deviceUuid**

A Unique ID belonging to the device that made this recording. Only applicable to Triangle/6L devices.

String **hardwareVersion**

The specific hardware version of the device that made this recording. Only applicable to Triangle/6L devices.

String **firmwareVersion**

The specific firmware version of the device that made this recording. Only applicable to Triangle/6L devices.

String **serialNumber**

The serial number of the device that made this recording. Only applicable to Triangle/6L devices.

Float **batteryLevel**

The battery level of the device that made this recording. Only applicable to Triangle/6L devices.

LeadConfiguration **leadConfiguration**

The lead configuration that was used for this recording

class RecordingConfiguration

Contains configuration data about a Recording. Used to configure **the new recordings**, and also is a property on the AliveCorEcg, use **getters** and **setters** as all fields are private.

Default values and enums that can be used for RecordingConfiguration:

```
public static final int MAINS_FREQUENCY_60Hz = 60;
```

```
public static final int MAINS_FREQUENCY_50Hz = 50;
```

```
public static final int DEFAULT_MAX_DURATION_SECONDS = 30;
```

```
public static final int DEFAULT_RESET_DURATION_SECONDS = 10;
```

```
public static final FilterType DEFAULT_FILTER_TYPE = FilterType.ENHANCED;
```

Properties	
AliveCorDevice device	The type of device that made this recording.
String deviceList	Supported device list
LeadConfiguration leadConfiguration	The lead configuration that was used for this recording,
FilterType filterType	The type of filter being used by this recording FilterType.ENHANCED default value, or FilterType.ORIGINAL
int maxDurationSeconds	The maximum duration of the recording, in seconds. For KaridaAI Instant Analysis, at least 30 seconds are required.
int resetDurationSeconds	The minimum duration, in seconds, before a recording can be saved. If the user loses contact with the device leads before this time, the recording will be restarted
int mainsFrequency	The Frequency for the Mains Filter to be used. This filter is responsible for eliminating signal noise caused by the nearby electrical power grid. It should be either 50 or 60, depending on your location.
boolean enableLeadsButtons	To disable and get leads switch status for Kardia 6L devices use next methods public void setEnableLeadsButtons (boolean enableLeadsButtons); public Boolean getEnableLeadsButtons (); Same as RecordEkgConstants.EXTRA_ENABLE_LEADS_BUTTONS
@Deprecated boolean hideTrace	Disable trace on the StripView, could be used some customers per request
@Deprecated String overrideDeviceName	You can set override DeviceName

class EcgEvaluation

Contains the findings from KardiaAI analysis, including the Determination and detected heart rate.

Properties

AliveCorDetermination **getDetermination()**

The final Determination of this evaluation

String **getKaiResult()**

The final Determination of this evaluation, in a lower-case string form.

String **getKardiaAiVersion()**

The version of KardiaAI used to analyze this recording.

Float **getAverageHeartRate()**

The average heart rate for this recording. Can be null if a heart rate could not be determined

boolean **isInverted()**

If true, KardiaAI determined that the recording's lead 1 was inverted. That is, the user may have had their left hand on the right lead, and right hand on the left lead. This causes the final ECG trace to appear upside down.

CharSequence **getAlgorithmResultText()**

A short, localized, human-readable description of the determination.

CharSequence **getAlgorithmResultDescription()**

A long-form description of the determination, localized and human-readable

CharSequence **getResultDisclaimerText()**

Some disclaimer text regarding the determination, localized and human-readable. Defined for next values: UNREADABLE, AFIB, NORMAL, BRADYCARDIA, TACHYCARDIA, UNCLASSIFIED.

enum AliveCorDetermination

Represents the final determination resulting from an ECG analysis

Kardia AI V1 values:	
NORMAL	A Normal Sinus Rhythm was detected
AFIB	Signs of Atrial Fibrillation were detected
UNCLASSIFIED	No relevant information could be obtained from analyzing this recording. It may have an analyzed heart rate, but nothing conclusive can be said about the final determination, good or bad.
TACHYCARDIA	Tachycardia was detected
BRADYCARDIA	Bradycardia was detected
SHORT	The recording was too short to be analyzed. A heart rate could not be conclusively determined.
LONG	The recording was too long to be analyzed. A heart rate could not be conclusively determined.
UNREADABLE	The recording was unreadable due to signal noise. A heart rate could not be conclusively determined.
NO_ANALYSIS	Analysis could not be performed on this recording. A heart rate could not be conclusively determined.
Kardia AI V2 values:	
SINUS_RHYTHM	Normal Sinus Rhythm
SINUS_RHYTHM_PACS	Sinus Rhythm with Supraventricular Ectopy
SINUS_RHYTHM_WIDE_QRS	Sinus Rhythm with Wide QRS
SINUS_RHYTHM_PVCS	Sinus Rhythm with Premature Ventricular Contractions

enum LeadConfiguration

Represents the configuration of the leads of an EKG recording. Recordings can be single-lead or six-lead

Values

SINGLE

The recording has a single lead

SIX

The recording has six leads.

com.alivecor.view

class EcgResultView

A View that can show rhythm strips for an ECG, as well as the Kardia AI Determination, pulse, and controls allowing inversion. The View takes care of all display logic and input-handling related to it. SDK clients simply need to set the **AliveCorEcg** to display.

Public Methods

```
public void setEcgRecord(AliveCorEcg ecg)
```

Sets the ECG that should be shown in this View. It will be loaded from disk if present.

class RhythmStripView

A View that can show rhythm strips for an ECG. It can be styled in a number of ways, and can be configured either as a scrolling view of the entire trace, or as a non-scrolling preview of it.

Public Methods

```
public void setEcgRecord(AliveCorEcg ecg)
```

Sets the ECG that should be shown in this View. It will be loaded from disk.

```
public void setAlwaysSingleLead(boolean alwaysSingleLead)
```

If true, the RhythmStripView will only show the first Lead of a recording, even if it is a 6-lead.

```
public void setDisplayMode(DisplayMode mode)
```

Sets the Display Mode for this View. The modes are FULL, which is a fully-scrollable view of the whole trace, or PREVIEW, which shows a non-scrollable piece in the middle of the trace.

```
public void setGraphScale(float scale)
```

Sets the scale for the graph. The trace and grid can be magnified or reduced using this scaling parameter.

```
public void setFilterType(FilterType filter)
```

Sets the filter to be used on the ECG trace being displayed. ORIGINAL or ENHANCED

```
public void setInverted(boolean inverted)
```

If true, the ECG Trace will be inverted along the Y axis: ie, upside-down

com.alivecor.ecg.record

class RecordEkgFragment

Presents the recording flow for an EKG. Will establish a connection with the chosen device and collect EKG data until a recording is completed. The Fragment displays UI feedback/handling for most error cases, though it provides listener callbacks for applications to provide additional handling.

Public Methods

```
public static RecordEkgFragment newInstance  
(RecordingConfiguration config) throws IllegalArgumentException
```

Creates a new instance of this Fragment for the given Recording Device. If the device described by parameter is not supported, or if AliveCorKit was not properly initialized, then this method will throw `IllegalArgumentException`

Parameters

recordingConfiguration: Configuration object describing the recording to perform

```
public static RecordEkgFragment newInstance
    (RecordingConfiguration config, RecordingHelpUrls urls) throws InvalidArgumentException
```

Creates a new instance of this Fragment for the given Recording Device. If the device described by parameter is not supported, or if AliveCorKit was not properly initialized, then this method will throw `InvalidArgumentException`

Parameters

recordingConfiguration: Configuration object describing the recording to perform

urls: URLs to replace for showing your own Help Page

```
public static RecordEkgFragment newInstance
    (RecordingConfiguration config, RecordingHelpUrls urls, boolean singleDevice) throws
    InvalidArgumentException
```

Creates a new instance of this Fragment for the given Recording Device. If the device described by parameter is not supported, or if AliveCorKit was not properly initialized, then this method will throw `InvalidArgumentException`

Parameters

recordingConfiguration: Configuration object describing the recording to perform

urls: URLs to replace for showing your own Help Page

singleDevice: If true, will not allow users to change their device (removes the menu option)

```
public void addListener(RecordEkgListener listener)
```

Adds a listener for events from this Fragment. Note that error callbacks do not represent permanent failures. The Fragment reports these error conditions for applications to do additional handling, but will attempt to resolve the situation on its own, possibly with user input or navigating away from the host Activity.

```
public void removeListener(RecordEkgListener listener)
```

Removes the previously-added `RecordEkgListener` from this Fragment. Regardless of whether or not there was a listener already added, calling this method will stop any ongoing recordings, and

reset the recording state. Call this no later than `onDetach` for Fragments or `onStop` for Activities

interface RecordEkgListener

This interface provides listener methods for `RecordEkgFragment`

Methods

```
void onRecordCompleted(AliveCorEcg evaluationResult)
```

Called when a recording is completed. The result of the recording, including KardiaAI integration, is included. After this call, the Fragment will perform no more operations. The recording has been completed and the Result is ready to display.

```
void onRecordError(RecordingError error)
```

Called when the Fragment encounters a non-fatal error. **These errors are not indicative of a failure to record.** The Fragment will attempt to resolve these errors, potentially with user input. This callback is provided for additional handling, and for informational purposes.

```
void onUserCancel()
```

Called when the user signals an intent to cancel recording. This can be handled by the listener to remove the Fragment

```
void onBTPairingCancel ()
```

Called when the user cancels Bluetooth pairing system dialog. This can be handled by the listener to show additional information to the user or cancel recording.

```
void onChangeDevice()
```

Called when the user signals an intent to change their recording device. Clients should present a UI allowing them to choose a different device, then replace this `RecordEkgFragment` with a new one for the newly-chosen device.

```
void onLeadConfigUpdated(LeadConfiguration config)
```

Called when the user signals an intent to change their lead configuration. This event is handled by the SDK, but a callback is provided for additional handling

```
void onRecordSettings(L)
```

Called when the user signals an intent to navigate to record settings. Client apps should launch their settings screen.

```
void onEcgFrame(LeadValues latest)
```

A callback (onEcgFrame) that an app can hook into to tee off the raw values coming from the device.

```
void onBPMUpdated(String bmp)
```

A callback for overridable Hook for additional handling of BPM updated values each frame.

enum RecordingError

This enum represents the errors that can be made during recording.

Methods

```
static RecordingError fromName(String name)
```

Given a string representing the name of a `RecordingError`, this method will return its enum value

Values

```
NFC_ON
```

The device's NFC hardware is turned on. NFC must be disabled in order to use AliveCor devices. The Fragment will present a UI which may direct a user out of the app in order to resolve this error

BLUETOOTH_OFF

The device's bluetooth hardware was powered down. The Fragment may direct the user to an external activity to resolve this issue.

MIC_PERMISSION_EKG

Certain AliveCor devices require the microphone in order for the device to collect data. If the `RECORD_AUDIO` permission is required, the Fragment will direct the user to a permission prompt, or possibly the Settings screen in order to resolve this error.

LOCATION_PERMISSION

On Android, BLE Devices like Triangle require location services to work. If the `FINE_LOCATION` permission is required, the Fragment will direct the user to a permission prompt or Setting screen in order to resolve this issue.

LOCATION_OFF

On Android, BLE Devices like Triangle require location services to work. If the user has previously granted location permission but turned off their location services, then the Fragment will direct to a prompt to enable them.

TRIANGLE_BATTERY

If you are attempting to record with a Triangle Device and the battery runs out during recording, this error will be thrown. The user will be notified and can attempt to reconnect or connect to a different Triangle device

MAINS_NOISE_EARLY

If excessive signal noise is detected during recording, this error will be thrown and the recording will be stopped. The User will be given an opportunity to retry the recording.

UNKNOWN

An exceptional error has occurred. Recording will attempt to start over from the beginning.

class ResultEkgFragment

The ResultEkgFragment presents the result of an analyzed recording, allowing the user to view the trace, the KardiaAI finding, etc.

Methods

```
public static ResultEkgFragment newInstance(AppCompatActivity activity,  
                                             RecordResult recordResult)
```

Creates a new instance of this Fragment, to show the given `RecordResult`

```
public void addListener(ResultEkgListener listener)
```

Adds a ResultEkgListener, to listen for events from this Fragment

```
public void removeListener(ResultEkgListener listener)
```

Removes the previously-added ResultEkgListener from this Fragment. Regardless of whether or not this Fragment had a listener attached, calling this method remove the recording result and clear all data from this Fragment.

Call this method no later than `onDetach` for a Fragment or `onStop` for an Activity.

interface ResultEkgListener

A listener interface for the `ResultEkgFragment`, allowing it to communicate relevant events to an Activity or other component

Methods

```
void onSuccess()
```

The user has completed their work with this Fragment. It would be a good time to remove it and move on to the next part of your UI flow

```
void onFailure(ResultError error)
```

This is called if there was an error during the Result flow.

enum ResultEkgError

Represents errors encountered in the Result Fragment

Values

```
WRONG_INPUT
```

A possible failure reason on the Result flow.

AliveCorKit Classes

com.alivecor.api

class AliveCorKit

Manages configuration and authentication with the AliveCorKit Library, and provides a facade for all AliveCorKit functionality

Methods

```
public static synchronized void initialize(@NonNull Context context,  
                                           String apiKey,  
                                           InitListener listener,  
                                           String appName,  
                                           String bundleId,  
                                           String partnerId,  
                                           boolean enableConsoleLogging)
```

Initializes the AliveCorKit library with the given API key, enabling console logging if requested. The library will use **AliveCor's Production API** for authentication and storage

Parameters

context: A Context. This Context's `applicationContext` will be retained and used for operations

appName: The name of your app. This will appear in the copy of certain strings in the premade UIs

apiKey: A valid AliveCor mobile API Key. Contact us if you need a key

enableConsoleLogging: If true, AliveCorKit will log to the console as it operates. If you are using Timber and install a tree, you will probably see our logs regardless of this value.

```
public static synchronized void initialize(@NonNull Context context,
                                         String apiKey,
                                         InitListener listener,
                                         String appName,
                                         String bundleId,
                                         String partnerId,
                                         AliveCorServer backendServer,
                                         boolean enableConsoleLogging)
```

Initializes the AliveCorKit library with the given API key, enabling console logging if requested. The library will use the specified backend server for authentication and storage

Parameters

context: A Context. This Context's `applicationContext` will be retained and used for operations

apiKey: A valid AliveCor mobile API Key. Contact us if you need a key

appName: The name of your app. This will appear in the copy of certain strings in the premade UIs, use a USER AGENT

server: The backend Server to use for auth and storage. You may choose PRODUCTION for our production API or STAGING for our development/testing API.

enableConsoleLogging: If true, AliveCorKit will log to the console as it operates. If you are using Timber and install a tree, you will probably see our logs regardless of this value.

```
public static AliveCorKit get()
```

Gets an instance of `AliveCorKit`. Do not call before calling `initialize()`

```
public KardiaProEcgBackend getEcgBackend()
```

Get a Backend object that can be used to store EKGs, authenticate with AliveCor's backend servers, etc. Further documentation of this object is below.

```
public Intent getRecordIntent(@NonNull AliveCorDevice device, String patientId)
```

Get an Intent that can launch the Default Flow. for the given device. To see what devices are supported, use `getSupportedDevices()`. Information about the result of the recording will be delivered by `onActivityResult()`, assuming the Activity is launched with `startActivityForResult()`.

Parameters:

device: The Device to use for recording. Valid choices depend on your API Key

patientId: A valid Patient ID for this Team. Patient IDs are obtained externally depending on the workflow for your team and your application.

The Record Activity can return the following result data:

Result Codes

`RESULT_OK` is returned if the UI flow ended with the user making and accepting a recording

`RecordEkgConstants.RESULT_CHANGE_DEVICE` is returned if the user indicates they wish to switch devices. Calling Activities should launch a UI to choose a different device, and relaunch with this new device.

`RESULT_USER_CANCEL` is returned if the user did not successfully make a recording during their time in the UI flow

Result Data

In most cases, the result Intent will contain data that can be unmarshalled using the `getRecordActivityResult()` method of this class. The `RecordActivityResult` will contain an object representing the successful recording, as well as any non-fatal errors encountered during the flow. These errors should not be considered evidence of failure. If the

recording was successful. The result code will be `RESULT_OK` and `RecordActivityResult#isSuccessful()` will return `true`.

```
@Nullable public RecordActivityResult getRecordActivityResult(@Nullable Intent data)
```

Parses the result of the Default Record Activity (obtained from `onActivityResult()`) and returns it. If the result intent did not contain any result data this method will return `null`

```
public EkgAnalyzer getClassifier()
```

Get a **EkgAnalyzer** that can be used to analyze recorded ECG data without a UI.

```
public String getVersion()
```

Returns the version of `AliveCorKit`

```
public List<AliveCorDevice> getSupportedDevices()
```

Returns a list of all devices supported by the library, once initialized with a valid API key. Attempts to begin recording with an unsupported device will result in Exceptions being thrown

class AliveCorKitUtil

AliveCorKitUtil provides some useful constants and utility methods for dealing with ECG data.

Constants

```
public static final String ECG_METADATA_PATIENT_MRN
```

Key on a `KardiaEcgEntity`'s metadata for the patient ID that made the recording

```
public static final String ECG_METADATA_TEAM_ID
```

Key on a `KardiaEcgEntity`'s metadata for the Team ID of the team that made the recording.

Methods

```
static KardiaEcgEntity entityFromRecordResult(@NonNull RecordResult result)
```

Maps a `KardiaEcgEntity` from a `RecordResult`, obtained from `RecordEkgFragment` or from the `result` field of the object obtained from

```
AliveCorKit.getRecordActivityResult(Intent).
```

```
static EcgWebEntity loadWebEntity(@NonNull KardiaEcgEntity from)
```

Creates an `EcgWebEntity` for POSTing the given `KardiaEcgEntity`. This method does disk I/O, as the `EcgWebEntity` contains the encoded bytes of the recording's ATC files.

com.alivecor.pro

enum AliveCorServer

Represents the server backend the AliveCorKit library should use when performing auth and storage. You may choose between our production environment or our staging/testing environment.

Values

```
STAGING_US
```

The Staging environment servers. This environment has its own database and can be used for development and testing purposes. The staging environment is not guaranteed to be reliable.

```
PRODUCTION_US
```

The Production environment servers. This environment is the live production server used for real-world customers.

Note: Starting from SDK 1.4.0 we added `STAGING_EU` and `PRODUCTION_EU` servers for future releases.

class EcgWebEntity

EcgWebEntity is a Gson-compatible class that represents the POST and response bodies from the AliveCor backend. You should not have to interact directly with this class if you are using `EcgRepository`

Properties

String **serverId**

The server-side ID of the entity

AlgorithmDetail **algorithmDetail**

Additional information about the KardiaAI algorithms used to analyze this recording.

String **originalAtcFileData**

A base-64-encoded string representing the entire Original ATC file

String **enhancedAtcFileData**

A base-64-encoded string representing the entire Enhanced-Filtered ATC file

float **bpm**

The analyzed heart rate of the recording

String **localId**

The local database ID of the recording

boolean **inverted**

True if the recording was inverted, false otherwise. For multi-lead recordings, this value is not valid.

String **dateString**

An RFC 3339-Compliant timestamp representing the time of the recording.

String **algorithmDetermination**

The final determination of KardiaAI analysis for this recording

class EcgWebEntity.AlgorithmDetail

Represents more detailed information about the algorithms that analyzed this recording.

Properties

String **determination**

The final determination of KardiaAI analysis for this recording

String **algoVersion**

The version of the algorithm used to analyze this recording

class KardiaEcgEntity

Represents a saved ECG entity in the `KardiaEcgRepository`.

Properties

String **localId**

The local ID of the recording

String **publicId**

The server-side ID of the recording. This can be null for recordings that have not yet been uploaded

String **finalDetermination**

The final determination from KardiaAI analysis of this recording

String **algorithmVersion**

The version of the KardiaAI algorithms used for this recording

DateTime **recordedAt**

The time and date of the recording. The time zone should be UTC, and will be converted to UTC before being stored in the database.

float **averageHeartRate**

The analyzed heart rate of this recording

boolean **isSixLead**

True if the recording is six-lead, false otherwise

boolean **isDirty**

Records marked as `dirty` must be uploaded or updated on the backend server. `KardiaProEcgRepository.uploadDirtyEcgs()` will upload these recordings.

Map<String,String> **metadata**

Map of key-value pairs provided for SDK clients' use. Additional information about recordings can be stored here for later use.

RecordingDeviceData **recordingDeviceData**

Data about the device that made this recording

class KardiaProEcgBackend

KardiaProEcgBackend provides AliveCorKit's storage, authentication, and REST functionality. An instance of this object can be obtained by calling `AliveCorKit.get().getEcgBackend()`

Methods

KardiaProEcgRepository **getRepository**

Gets an instance of `KardiaProEcgRepository` that can be used to manage EKG data with the backend or locally. Documentation for this object is elsewhere in the document.

boolean **validatePatientId**(String patientId)

Validates a patient ID against the configured pattern for the logged-in user's Team.

Completable **loginWith**(String username, String password)

Subscribe to this Completable to authenticate with the Kardia Pro backend. Once authenticated, this Completable will complete. If an error occurred, it will be reported to the subscriber.

Completable **logout**()

Logs the currently-logged-in user out and deletes all their Team Member information.

TeamMember **getOwnTeamMember**()

edGet a `TeamMember` object representing the currently-logged-in user. If there is no one logged in, then this method will return null.

boolean **isLoggedIn**()

Returns true if there is a valid login session active.

KardiaPublicWebapp **getWebapp**()

Gets a `KardiaProPublicWebapp` object, which can be used for doing REST calls directly against AliveCor servers. Note that `KardiaProEcgRepository` provides better, higher-level functionality for REST calls. The webapp is provided for special requirements and cases.

class `KardiaProEcgRepository`

Repository class for doing high-level storage and REST operations on ECG records. Records can be stored locally as long as this Object can be accessed, but REST operations require a user to be logged in. (see `KardiaProEcgBackend.loginWith()`)

Methods

`Single<KardiaEcgEntity> writeEcgRecord(KardiaEcgEntity entity)`

Writes the given `KardiaEcgEntity` to the database and emits it back. If the record was already in the database, it will be replaced with this new record.

`Flowable<KardiaEcgEntity> getAllLocalEcgRecords()`

Emits all locally-stored recording objects.

`Flowable<KardiaEcgEntity> getAllDirtyEcgRecords()`

Emits all locally-stored recording objects that need to be uploaded.

`Maybe<KardiaEcgEntity> getEcgRecord(String ecgLocalId)`

Reads a `KardiaEcgEntity` from the database and emits it. If no entity with that local ID exists, this will complete without emitting anything.

`Flowable<KardiaEcgEntity> uploadDirtyEcgs()`

Uploads all dirty ECGs to the AliveCor backend. Each uploaded entity will be updated in the database with refreshed data (if available) and will be emitted. A user must be logged in in order for this to succeed.

`Single<KardiaEcgEntity> uploadEcgRecord(KardiaEcgEntity entity)`

Uploads the given `KardiaEcgEntity` to the AliveCor backend. A user must be logged in in order for this to succeed. The record will be written to the database, regardless of upload success or failure. If upload failed, it will be saved as `dirty` so upload can be re-attempted.

class TeamMember

A Gson-compatible class representing a Kardia Pro team member. The currently-logged in user is represented as a TeamMember

Properties

TeamMember.Profile **profile**

The user profile information for the Team Member

List<TeamMember.Team> **teams**

A list of all the Teams the user is a member of

class TeamMember.Profile

Represents the user profile of a team member. The user's name, ID, and permissions can be found here.

class TeamMember.Team

Represents a Team of which a team member could be a part. The team's name, avatar and logo URLs, and screening configuration can be found here.

Class TeamMember.Team.ScreeningConfig

Represents the screening configuration for this team.

Properties

String **patientInputidValidationRegex**

A regular expression for which a Team's patient IDs must conform.